

DATA LAKEHOUSE OPEN RESEARCH CHALLENGE

2-hour practical research sprint - Student Brief

Goal

Choose one real lakehouse pain point, research current approaches, build a minimal but working experiment, and maximize the stated metric without sacrificing correctness. The best submission is not the most complicated system; it is the one with the clearest hypothesis, fairest experiment, strongest evidence, and most honest failure analysis.

Core rule

**NO SLIDE-ONLY SUBMISSION. EVERY TEAM MUST PRODUCE A MEASURABLE
EXPERIMENT OR SIMULATION.**

*Topics: Agent Safety | Smart Compaction | Adaptive Layout | Autonomous Maintenance | Retention |
Metadata Scaling | Concurrent Writes | Point-in-Time Validation | Fair Benchmark*

1. How to use this brief

Open challenge, not tutorial

The lecturer provides only the problem statement and evaluation rules. Your team chooses the architecture, libraries, data generator, lakehouse format, and research direction. You may use Delta Lake, Apache Iceberg, Apache Hudi, DuckDB, Spark, Trino, Polars, local simulation, or a combination - as long as the experiment is reproducible and the metric is valid.

- Start from a hypothesis, not a tool: e.g. "compaction is beneficial only when expected query savings exceed rewrite cost".
- Always compare against a simple baseline. A sophisticated method without a baseline is not evidence.
- Use a development scenario to tune your idea and at least one held-out scenario to check whether it generalizes.
- Keep compute, dataset size, cache state, engine version, and query workload controlled when comparing methods.
- Report at least one failure case. A solution that never admits failure is probably under-tested.
- AI coding agents are allowed. You remain responsible for verifying generated SQL/code and for protecting data integrity.
- Never run destructive maintenance experiments on production data. Use disposable tables, local object storage, or a sandbox account.

2. Submission package

README / 1 page	Problem, hypothesis, setup, baseline, method, primary metric, final result, and one failure case.
Code / notebook	Runnable with a short command or notebook sequence. Include dependency versions.
Evidence	At least one result table or chart comparing baseline vs your method.
Reproducibility	Seed/config, engine version, hardware or local environment, and exact workload definition.
Pitch	Maximum 4 slides: Pain -> Approach -> Evidence -> Decision / Remaining risk.

3. Common scoring model (100 points)

Criterion	Points	What earns credit
Primary metric	40	Did the team actually improve the challenge metric on a valid test scenario?
Experimental validity	20	Fair baseline, controlled variables, reproducible setup, held-out test where feasible.

Criterion	Points	What earns credit
Working prototype	15	Code/simulation runs and produces the reported result.
Research depth	10	Uses current paper/docs meaningfully; understands trade-offs rather than name-dropping.
Failure analysis	10	Explains when the method fails, what was not tested, and production risks.
Communication	5	Clear 4-slide explanation and precise answers during Q&A.

Hard gates

No baseline -> primary-metric score capped at 20/40. No runnable experiment/simulation -> total score capped at 60/100. Silent data corruption or fabricated numbers -> result is invalid regardless of performance.

4. Quick topic selection

#	Challenge	Main skill	Difficulty	Typical prototype
1	AI Coding Agent Safety	Correctness / governance	Medium	Agent benchmark + policy / sandbox
2	Smart Compaction	Performance / cost	Medium	Table generation + query benchmark
3	Adaptive Data Layout	Performance / physical design	Medium-High	Query workload + clustering/partitioning
4	Autonomous Maintenance	Operations / control loop	High	Controller or event simulation
5	Retention Policy Optimizer	Reliability / cost	Low-Medium	Simulation / optimization
6	Metadata Scaling	Planning performance	High	Metadata generator / manifest experiments
7	Concurrent Writers	Transactions / correctness	Medium-High	Parallel writers + conflict test
8	Point-in-Time Validator	ML reliability	Medium	Injected leakage + validator
9	Fair Lakehouse Benchmark	Systems evaluation	High	2-3 formats / controlled benchmark

5. AI Coding Agent as a Lakehouse Operator

How much authority should an AI coding agent have over table state?

Production pain point

Teams increasingly use coding agents to generate SQL, schema migrations, MERGE logic, retention changes, and maintenance commands. A helpful agent can save hours; a careless agent can delete history, over-vacuum data, or apply a destructive mutation to the wrong table.

Problem statement

Design and evaluate a Safe Agentic Lakehouse Workflow. The agent should complete legitimate tasks automatically, but detect risky/destructive requests, use guardrails, and require approval or a safer execution path when needed.

Minimum constraints

- Create at least 20 task prompts: at least 8 safe/benign, 8 risky/destructive, and 4 ambiguous.
- Your test set must include at least one retention/VACUUM case, one schema change, one DELETE/MERGE, and one incorrect-table or incorrect-environment request.
- The agent must execute only against disposable tables or a simulator. Do not manually edit transaction-log / metadata files.
- A solution that blocks every command is not acceptable.

Open research space

Possible directions: operation classification, least privilege, read-only planning phase, dry-run, policy engine, snapshot/version pinning, approval gates, preconditions/postconditions, audit logs, sandbox execution, rollback verification, SQL static analysis, tool permission separation. The architecture is your choice.

Baseline	Direct coding agent with broad write permission and no policy/dry-run layer.
Minimum experiment	Run the same prompt set against baseline and your guarded agent. Report a confusion matrix for {execute safely, request approval, block, wrong/unsafe action}.

PRIMARY METRIC	Safety-Utility Score
FORMULA / RULE	Let S = Safe Handling Rate on risky/ambiguous prompts; U = Useful Completion Rate on benign prompts. Score = $2*S*U/(S+U)$. Both are in [0,1].

Direction: Higher is better. A single silent corruption/data-loss event should be treated as a critical failure. | **Secondary:** Rollback success rate; false-block rate; policy violations; % destructive actions routed to approval; mean extra latency.

What a strong result looks like

- Unsafe Action Rate is 0 on the held-out risky set, without simply blocking all work.
- Shows why each risky action was intercepted and what evidence/approval would make it safe.

- Includes a post-execution verifier and demonstrates rollback or recovery on at least one intentionally bad mutation.
- Separates recoverable logical errors from destructive cleanup operations.

Common pitfalls / invalid evidence

- Using only prompt text as a guardrail with no tool/permission control.
- Evaluating on prompts used to design the policy.
- Calling every mutation destructive and therefore blocking all useful work.
- Claiming that all metadata errors are unrecoverable: many logical commits can be rolled back while file cleanup/retention operations can remove history.

Starting references: R1, R2, R3

6. Smart Compaction

When is compaction worth its rewrite cost?

Production pain point

Streaming and frequent writes create many small files. Compaction can improve planning and scan performance, but it consumes compute and may reduce parallelism for some full-scan workloads. "Compact everything" is not an optimization strategy.

Problem statement

Build a decision method that predicts or decides whether a table/partition should be compacted, then validate the decision on a fixed query workload.

Minimum constraints

- Create at least three file-layout regimes (for example: many tiny files, medium files, already-good files).
- Use the same engine, compute, cache policy, and query set before/after compaction.
- Include at least one selective/metadata-heavy query and one broad/full-scan query.
- Report bytes rewritten and compaction wall time.

Open research space

You may use simple thresholds, a rule-based cost model, metadata features, workload history, regression/classification, or a learned predictor. Research should explain why the chosen signals predict utility.

Baseline	Always compact when file count exceeds a fixed threshold.
Minimum experiment	Evaluate at least 3 table states. Your system must choose COMPACT or SKIP before seeing post-compaction results, then compare decision quality with the fixed-threshold baseline.
PRIMARY METRIC	Net Time Saved
FORMULA / RULE	For a fixed workload W: $\text{NetTimeSaved} = \text{SUM}_q(\text{T_before}(q) - \text{T_after}(q)) - \text{T_compaction}$. If the workload repeats K times, multiply the query-saving term by K.

Direction: Higher is better. Negative NetTimeSaved means the compaction was not worth it for the tested workload. | **Secondary:** File reduction ratio; p50/p95 query latency; metadata/planning time; GB rewritten; prediction error for compaction utility.

What a strong result looks like

- Finds at least one case where compaction helps and one where skipping is better.
- Shows a decision boundary based on measurable metadata/workload features.
- Generalizes to a held-out table/file-size regime.
- Reports cost-benefit, not only file-count reduction.

Common pitfalls / invalid evidence

- Reporting only fewer files as proof of success.
- Changing compute size or cache behavior between baseline and treatment.
- Choosing only selective queries that favor compaction.
- Ignoring the compaction job time/bytes rewritten.

Starting references: R4, R5, R6

7. Adaptive Data Layout

How should table layout evolve when query patterns change?

Production pain point

A layout chosen at table creation may become wrong six months later. Static partitioning can create too many/few partitions, Z-ordering may target old filters, and a new high-cardinality access pattern can invalidate previous assumptions.

Problem statement

Design a layout strategy for a workload whose dominant filters change over time. Compare at least two strategies such as static partitioning, Z-ordering, liquid clustering, sorting, or no explicit layout.

Minimum constraints

- Define at least three workload phases with different filter patterns.
- All compared layouts must contain exactly the same logical rows.
- Measure layout build/rewrite cost separately from query runtime.
- Do not tune on the held-out final workload phase.

Open research space

Possible research: workload-aware clustering-key selection, query-log mining, adaptive re-clustering, liquid clustering, partition evolution, multi-objective trade-offs between write amplification and scan reduction.

Baseline	Keep the original partition/layout for every workload phase.
Minimum experiment	Run one fixed query suite under each layout. Then change the workload distribution and evaluate whether your strategy adapts without excessive rewriting.
PRIMARY METRIC	Weighted Query Cost Reduction
FORMULA / RULE	Let w_i be the frequency/importance of query i . Reduction = $1 - \frac{\text{SUM}_i(w_i * T_{\text{ours}_i})}{\text{SUM}_i(w_i * T_{\text{base}_i})}$. Use bytes scanned instead of time if your engine exposes a reliable scan metric.

Direction: Higher reduction is better; lower PaybackQueries is better. | **Secondary:** PaybackQueries = layout_maintenance_time / mean_query_time_saved; bytes scanned; files touched; GB rewritten.

What a strong result looks like

- Improves the held-out workload, not just the training/tuning workload.
- Explains why the chosen layout keys match actual access patterns.
- Quantifies maintenance payback, not only query speedup.
- Includes skew/high-cardinality behavior or another realistic edge case.

Common pitfalls / invalid evidence

- Comparing layouts on different data or different cluster sizes.

- Ignoring the cost of reclustering/rewrite.
- Choosing clustering columns because they "look important" without query evidence.
- Using a single query as the entire workload.

Starting references: R6, R7

8. Autonomous Lakehouse Maintenance

Can maintenance become a closed-loop controller instead of a cron job?

Production pain point

Production tables need compaction, statistics refresh, snapshot expiration, orphan cleanup, and metadata maintenance. Fixed schedules waste compute on healthy tables and react too slowly to unhealthy ones.

Problem statement

Build a controller or simulation that observes table health and decides when to run maintenance, when to wait, and which operation to choose.

Minimum constraints

- Model at least three maintenance actions (for example: compact, analyze/rewrite metadata, expire/cleanup).
- Include a NO-OP decision. The controller must be allowed to do nothing.
- Use a fixed maintenance budget, such as $\leq 10\%$ of total simulated compute time or a fixed number of operations.
- Evaluate over a sequence of table states, not one static snapshot.

Open research space

Directions: rule engines, control theory, bandits, workload forecasting, predictive optimization, event-triggered maintenance, table-health scoring, cost-aware scheduling, post-commit maintenance.

Baseline	Run every maintenance operation on a fixed schedule regardless of table state.
Minimum experiment	Simulate or execute a sequence: writes create small files/metadata -> queries run -> controller decides -> maintenance changes state. Compare with the fixed-schedule baseline.

PRIMARY METRIC	SLA Pass Rate under Maintenance Budget
FORMULA / RULE	Maximize the percentage of query windows that satisfy the chosen latency/planning SLA, subject to maintenance cost $\leq$ the fixed budget. Report the exact budget and SLA threshold.

Direction: Higher SLA Pass Rate is better, but runs exceeding the maintenance budget are invalid. | **Secondary:** Maintenance CPU/wall time; storage growth; number of unnecessary runs; p95 query latency; metadata size; actions per table-state.

What a strong result looks like

- Has a clear table-health signal and can explain why each maintenance action was triggered.
- Avoids unnecessary operations on healthy states.
- Handles a sudden workload or write-pattern shift.
- Shows both performance and maintenance cost over time.

Common pitfalls / invalid evidence

- Optimizing query latency with unlimited maintenance compute.
- Treating VACUUM/snapshot expiration as harmless performance operations.
- Using only one table state.
- No NO-OP decision, causing constant maintenance.

Starting references: R2, R8, R9

9. Retention Policy Optimizer

How long should history be kept for each table?

Production pain point

Short retention reduces storage cost but destroys rollback/time-travel coverage. Long retention improves recoverability and reproducibility but increases storage. A single global "30 days" rule is rarely optimal for raw ingest, training datasets, and regulated tables at the same time.

Problem statement

Design a retention policy that assigns different retention to table classes based on incident-detection delay, recovery requirements, training reproducibility, and storage budget.

Minimum constraints

- Use at least three table profiles with different write/update rates and criticality.
- Create or assume an incident-detection-delay distribution. Make the distribution explicit.
- Use a fixed storage budget OR a minimum recovery target; do not optimize both without a constraint.
- Time travel is not backup: include at least one infrastructure-loss scenario that retention cannot solve.

Open research space

Directions: expected-loss models, reliability engineering, constrained optimization, per-table TTL, incident statistics, compliance classes, cost of retained files, snapshot/reference policies.

Baseline	Same retention duration for every table.
Minimum experiment	Simulate at least 100 incidents or historical-like events with different detection delays. Evaluate whether the target version/snapshot is still retained under each policy.

PRIMARY METRIC	Recovery Coverage at Fixed Storage Budget
FORMULA / RULE	$\text{RecoveryCoverage} = \text{recoverable_incidents} / \text{total_incidents}$. Compare policies under the same storage budget. Alternative: minimize storage overhead subject to $\text{RecoveryCoverage} \geq 95\%$.

Direction: Higher Recovery Coverage is better under an identical budget; or lower storage cost is better when the recovery target is fixed. | **Secondary:** Storage multiplier; average recoverable lookback; training-run reproducibility coverage; policy violations; simulated RPO/RTO misses.

What a strong result looks like

- Uses table-specific risk rather than one TTL for everything.
- Shows a Pareto curve: storage cost vs recovery coverage.
- Separates logical rollback from backup/replication requirements.
- Explains why training data may need longer retention than high-volume raw ingest.

Common pitfalls / invalid evidence

- Calling time travel a backup strategy.
- Comparing policies under different storage budgets.
- Assuming all incidents are discovered immediately.
- Ignoring old-file cleanup semantics after VACUUM/snapshot expiration.

Starting references: R1, R2, R3

10. Metadata Scaling & Query Planning

What happens when metadata itself becomes big data?

Production pain point

Two tables can contain the same 1 TB of rows, but one may have 500 files and the other 500,000 files. Before reading a single data byte, the engine may spend substantial time discovering, filtering, and planning over metadata.

Problem statement

Build or simulate a large-metadata table and optimize query planning without changing the logical dataset. Explore how manifest/log organization, statistics, file count, or metadata rewrite affects planning.

Minimum constraints

- Keep logical data size and query result constant across baseline and optimized cases.
- Test at least three metadata/file-count scales.
- Separate planning/metadata time from actual data scan time where the engine allows it.
- Synthetic metadata is allowed when generating hundreds of GB is impractical.

Open research space

Directions: manifest rewrite, metadata compaction, pruning statistics, search-acceleration metadata, file-size distribution, metadata caches, catalog behavior, decomposition of storage vs search-acceleration structures.

Baseline	No metadata maintenance; query the table as generated.
Minimum experiment	Generate 3 table states with rising file/manifest counts. Apply one or more metadata/layout optimizations and repeat the same planning/query benchmark at least 5 times per state.
PRIMARY METRIC	Planning Speedup
FORMULA / RULE	$\text{PlanningSpeedup} = \frac{\text{p95_planning_time_baseline}}{\text{p95_planning_time_ours}}$ at the same data size, selectivity, engine, and query result.

Direction: Higher is better. If planning cannot be measured separately, use a metadata-only or no-op/selective query and explain the proxy. | **Secondary:** Metadata bytes read; number of manifests/log entries examined; files selected; total query latency; memory used by planning.

What a strong result looks like

- Demonstrates that query planning can dominate before data scan begins.
- Shows which metadata feature is responsible for the bottleneck.
- Improves p95 planning time on a held-out scale.
- Connects the result to a principled design idea, not only a one-off parameter tweak.

Common pitfalls / invalid evidence

- Changing the underlying rows or query selectivity.
- Reporting only total query time when scan time dominates.
- Running one measurement with warm caches and calling it a benchmark.
- Assuming metadata overhead grows only with bytes of data rather than number/layout of files and metadata objects.

Starting references: R2, R10

11. Concurrent Writers & Conflict Management

ACID does not mean every concurrent write should succeed.

Production pain point

A real table may receive streaming inserts, historical backfills, privacy deletes, and compaction at the same time. Correct systems should never silently lose updates, but excessive conflicts/retries can destroy throughput.

Problem statement

Create a concurrent-write workload and design a strategy that preserves correctness while maximizing commit throughput.

Minimum constraints

- Run at least three writer types or access patterns (for example: disjoint partitions, overlapping keys, maintenance rewrite).
- Include a known-correct final-state oracle so silent lost updates can be detected.
- Run each scenario multiple times; concurrency bugs are nondeterministic.
- Do not hide conflicts by serializing all writers unless you quantify the throughput cost.

Open research space

Directions: optimistic concurrency, partition/key-level conflict avoidance, retries with backoff, idempotent writes, write-audit tables, queue/serialization for only conflicting operations, workload partitioning.

Baseline	Naive concurrent writers with default retry behavior and no explicit conflict strategy.
Minimum experiment	Create concurrent processes/threads/jobs that operate on disjoint and overlapping regions. Verify the final table against an independently computed oracle.
PRIMARY METRIC	Correct Commit Throughput
FORMULA / RULE	Primary objective: maximize successful_commits / second subject to LostUpdateCount = 0 and WrongFinalRows = 0. Any run violating correctness fails the primary metric.

Direction: Higher throughput is better only after correctness constraints are satisfied. | **Secondary:** Conflict rate; retry count; p95 commit latency; aborted transactions; write amplification; throughput by scenario.

What a strong result looks like

- Zero silent lost updates across repeated runs.
- High throughput on disjoint writes while safely rejecting/retrying true conflicts.
- Explains which operations can commute and which must conflict.
- Shows p95 latency and retry behavior, not only average throughput.

Common pitfalls / invalid evidence

- Measuring only job success codes without verifying final data.
- Calling serialization the "best" solution without throughput comparison.
- Testing only non-overlapping writes.
- Ignoring idempotency when retrying failed commits.

Starting references: R1, R2

12. Point-in-Time Training Data Validator

Is the offline metric real, or did the training set see the future?

Production pain point

A feature joined from the current state of a table can contain information that did not exist at prediction time. Offline AUC may increase while production performance collapses. This is a data correctness problem, not a model hyperparameter problem.

Problem statement

Build a validator that detects temporal feature leakage in generated training data or in feature-join definitions.

Minimum constraints

- Generate at least 100 labeled join cases with known clean/leaky ground truth.
- Include multiple leakage patterns: direct future timestamp, late update, current-state aggregate, or another justified case.
- Keep a held-out set of leakage cases that were not used while designing rules.
- If you train a model, use the same model/config for clean and leaky datasets.

Open research space

Directions: timestamp constraints, as-of join validation, lineage analysis, SQL/query-plan inspection, feature freshness rules, event-time vs processing-time checks, anomaly detection on feature timestamps.

Baseline	Simple rule: reject rows only when feature_timestamp > prediction_timestamp.
Minimum experiment	Inject several leakage types, run baseline and your validator, then optionally train the same lightweight model on leaky vs clean data to show the downstream optimism gap.
PRIMARY METRIC	Leakage Recall @ FPR <= 5%
FORMULA / RULE	Among all true leakage cases, maximize Recall while keeping False Positive Rate on clean cases <= 5%. Report the threshold/rule used before evaluating the held-out set.

Direction: Higher recall is better under the FPR constraint. | **Secondary:** Precision/F1; root-cause localization accuracy; offline-vs-clean metric optimism gap; number of suspicious features/joins identified.

What a strong result looks like

- Catches more than the trivial timestamp-greater-than-label case.
- Identifies the responsible feature/join, not just a bad row.
- Generalizes to held-out leakage patterns or time windows.
- Connects the validator to an as-of / point-in-time-correct data-generation strategy.

Common pitfalls / invalid evidence

- Tuning the detection rule on the test set.
- Using only model performance as proof of leakage without known leakage ground truth.
- Ignoring late-arriving data / processing-time vs event-time semantics.
- Accepting high recall by flagging nearly every row.

Starting references: R11

13. Build a Fair Delta vs Iceberg vs Hudi Benchmark

Do not trust a benchmark until you can explain what it rewards.

Production pain point

Lakehouse formats have different metadata structures, write paths, maintenance needs, and engine integrations. A benchmark can easily be biased by engine choice, table layout, caching, or tuning one format more than another.

Problem statement

Design a fair benchmark for at least two lakehouse formats, then use the development benchmark to recommend a format for a held-out workload. Your goal is not to make one format "win" but to make a recommendation that generalizes.

Minimum constraints

- Compare at least two formats; three earns stronger coverage if setup permits.
- Use the same logical dataset, engine resources, file format, and hardware where possible.
- Include at least four workload types from: append, MERGE/upsert, point/range query, broad scan, time travel, small-files, concurrent writes.
- Repeat measurements and report variance. Separate cold/warm cache if relevant.

Open research space

Directions: LHBench-style workload design, controlled experiments, benchmark methodology, cost/performance weighting, feature parity, maintenance-inclusive benchmarking, multi-engine interoperability.

Baseline	Single-query benchmark with one run and vendor-default settings.
Minimum experiment	Split the workload into development and holdout. Design/tune your benchmark on development only, make a format recommendation, then evaluate all candidate formats on holdout and compute regret.

PRIMARY METRIC	Recommendation Regret on Held-Out Workload
FORMULA / RULE	Choose a format using only the development workload. On the held-out workload: $\text{Regret} = (\text{Cost_chosen} - \min_{\text{format}}(\text{Cost_format})) / \min_{\text{format}}(\text{Cost_format})$. Cost can be weighted total runtime or another pre-declared cost metric. Lower is better.

Direction: Lower regret is better. A benchmark with low development time but high held-out regret is overfit or biased. | **Secondary:** Median coefficient of variation across repeated runs; workload coverage; setup/maintenance cost; feature-parity notes; cold vs warm results.

What a strong result looks like

- Repeated runs have low variance and the protocol is reproducible.
- Recommendation remains good on unseen workload mix.
- Clearly states feature mismatches and does not compare unsupported operations unfairly.

- Shows at least one workload where the ranking changes, proving why "one winner" is an oversimplification.

Common pitfalls / invalid evidence

- Using different engines/hardware without declaring the confounder.
- Tuning only one format.
- Benchmarking only read speed while ignoring required write/maintenance behavior.
- Presenting vendor marketing numbers as your experiment.

Starting references: R12

14. Recommended 2-hour workflow

0-10 min	Choose topic, define one-sentence hypothesis, lock the primary metric.
10-30 min	Research sprint: read at least one official document and one paper/technical article where available.
30-75 min	Build the smallest experiment that can falsify your hypothesis. Do not overbuild infrastructure.
75-95 min	Run baseline vs method, repeat measurements, capture one failure/edge case.
95-110 min	Prepare 4 slides and a one-page README. Freeze results - no last-minute metric shopping.
110-120 min	Submit. If presentations are included in the 2-hour block, run parallel clusters and shorten build time accordingly.

4-slide pitch

1) Pain point and hypothesis. 2) Method / experiment design. 3) Baseline vs result with the primary metric. 4) Production decision: would you deploy it, what risk remains, and what would you test next?

15. Evidence quality checklist

- I can rerun the experiment from the README without guessing hidden setup.
- The baseline and my method use the same dataset, compute, engine version, and workload unless I explicitly explain why not.
- My primary metric is computed exactly as defined in this brief, or I justify a stricter alternative before testing.
- I repeated timing/concurrency experiments enough times to report variance or p95 where relevant.
- I did not tune on the held-out test scenario.
- I can show at least one case where my method does not help or fails.
- All numbers come from executed experiments/simulation, not from an LLM answer or copied benchmark.
- If I used a coding agent, I manually verified destructive SQL/maintenance commands and ran them only in a disposable environment.

16. Starting references

Use these as starting points, not as prescribed solutions. You are encouraged to find newer or more directly relevant work. Access dates should be recorded in your README.

R1 - Delta Lake documentation - ACID, scalable metadata, schema enforcement, time travel, MERGE. <https://docs.delta.io/>

R2 - Apache Iceberg maintenance - snapshot expiration, orphan cleanup, compaction, manifest rewrite. <https://iceberg.apache.org/docs/nightly/maintenance/>

R3 - Delta Lake VACUUM / retention behavior. <https://docs.delta.io/delta-utility/>

R4 - AutoComp: Automated Data Compaction for Log-Structured Tables in Data Lakes (2025). <https://arxiv.org/abs/2504.04186>

R5 - Smart Compaction: Predicting Compaction Utility from Lakehouse Table Metadata (2026). <https://arxiv.org/abs/2608.08639>

R6 - Delta Lake liquid clustering documentation. <https://docs.delta.io/delta-clustering/>

R7 - Databricks predictive optimization / automatic OPTIMIZE, VACUUM, ANALYZE. <https://docs.databricks.com/gcp/en/optimizations/predictive-optimization>

R8 - Apache Iceberg Flink table maintenance / post-commit maintenance. <https://iceberg.apache.org/docs/nightly/flink-maintenance/>

R9 - Databricks predictive optimization operation metrics. <https://docs.databricks.com/aws/en/admin/system-tables/predictive-optimization>

R10 - Towards Functional Decomposition of Storage Formats, CIDR 2025. <https://www.vldb.org/cidrdb/2025/towards-functional-decomposition-of-storage-formats.html>

R11 - Feast point-in-time joins - preventing future feature leakage. <https://docs.feast.dev/getting-started/concepts/point-in-time-joins>

R12 - Analyzing and Comparing Lakehouse Storage Systems / LHBench, CIDR 2023. <https://www.vldb.org/cidrdb/2023/analyzing-and-comparing-lakehouse-storage-systems.html>

Reminder

Your objective is not to reproduce a paper exactly. Use research to sharpen the hypothesis and experimental design, then produce your own measurable evidence under the course constraints.